

TRABALHO PRÁTICO

Infraestrutura Moderna com Docker, MySQL, Nginx e Wireshark

Disciplina: Segurança de Redes / Infraestrutura de Sistemas

Valor: 4,0 pontos | Individual ou grupos de até 4 alunos

IDENTIFICAÇÃO DA EQUIPE

Campo	Informação
Aluno 1 – Nome Completo:	Luiz Arthur Rufino Rodrigues
Aluno 1 – Matrícula:	202311023
Aluno 2 – Nome Completo:	Marcos Vinícius Ribeiro Gonçalves
Aluno 2 – Matrícula:	202411100
Aluno 3 – Nome Completo:	Rodrigo Azevedo Nunes
Aluno 3 – Matrícula:	201710561
Aluno 4 – Nome Completo:	Leonardo de Mendonça Lima
Aluno 4 – Matrícula:	202520551

VISÃO GERAL DO TRABALHO

Requisitos Funcionais

#	Requisito	Tecnologia
R1	API backend com cadastro e listagem de usuários	Node.js / Python / PHP / etc.
R2	Banco de dados relacional em container	MySQL
R3	Página web simples consumindo a API	HTML / JS / CSS
R4	Proxy reverso configurado	Nginx
R5	Orquestração completa do ambiente	Docker Compose
R6	Análise de tráfego da aplicação	Wireshark
R7	Documentação da arquitetura desenvolvida	Este documento (PDF final)

CRITÉRIOS DE AVALIAÇÃO (Total: 4,0 pontos)

Critério / Entrega	Descrição	Pts
1. Arquitetura e Containers (docker-compose.yml)	Ambiente sobe com docker-compose up. Containers (API, MySQL, Nginx) operacionais. Variáveis de ambiente e volumes configurados.	0,8
2. API Backend (CRUD de usuários)	Rota POST /users cria usuário no banco. Rota GET /users lista usuários. Retorno em JSON com status HTTP corretos.	0,8
3. Front-end + Nginx (proxy reverso)	Página HTML com listagem e formulário. Nginx redireciona /api → API e / → front-end. Configurações corretas no nginx.conf.	0,8
4. Análise Wireshark (captura de tráfego)	Capturas demonstram requisições HTTP entre serviços. Identificação de portas, protocolos e fluxo. Comentário sobre ao menos 3 pacotes distintos.	0,8
5. Documentação PDF (este documento preenchido)	Todos os campos obrigatórios preenchidos. Diagrama de arquitetura incluído. Links do Git e do vídeo válidos e acessíveis.	0,8
TOTAL		4,0

1. DESCRIÇÃO DA ARQUITETURA DO SISTEMA

Descrição textual da arquitetura:

O sistema é composto por um frontend, uma API desenvolvida em Flask, um servidor Nginx e um banco de dados MySQL. O usuário acessa a aplicação por meio do endereço localhost, sendo as requisições recebidas pelo Nginx, que entrega os arquivos do frontend e encaminha as chamadas da rota /api para a API Flask.

A API é responsável pelo cadastro e pela listagem de usuários, realizando a comunicação com o banco de dados MySQL para armazenar e recuperar informações. Durante a execução da aplicação, o Wireshark pode ser utilizado para capturar e analisar o tráfego de rede entre os componentes do sistema.



2. DESCRIÇÃO DAS PORTAS UTILIZADAS

Serviço / Container	Porta Interna	Porta Externa (Host)	Finalidade
Nginx (proxy reverso)	80	80	Entrada HTTP pública
API Backend	5000	Não exposta	Realizar requisições de acordo com a ação do usuário
MySQL	3306	Não exposta	Banco de dados
Frontend	127.0.0.1	127.0.0.1	Enviar solicitações do usuário

3. EXPLICAÇÃO DOS CONTAINERS

Container: Nginx (proxy reverso)

Imagem base:	nginx:alpine
Variáveis de ambiente:	Não tem
Volumes:	./frontend:/usr/share/nginx/html:ro ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
Dependências (depends_on):	api
Observações:	O navegador não acessa mais a porta 5000, o proxy reverso serve como intermediador enviando as requisições do site para a api por meio do caminho /api e não expõe a porta por meio da URL.

Container: API Backend

Imagem base / linguagem:	Python flask
Variáveis de ambiente:	Não tem
Volumes:	Não têm
Dependências (depends_on):	mysql
Rotas implementadas:	POST /users GET /users
Observações:	A conexão com a api passa por um healthcheck que faz com que a conexão inicie corretamente

Container: MySQL

Imagem base:	mysql:8.0
MYSQL_ROOT_PASSWORD:	root
MYSQL_DATABASE:	appdb
MYSQL_USER / PASSWORD:	Root / root
Volume de dados:	- ./init.sql:/docker-entrypoint-initdb.d/init.sql - mysql_data:/var/lib/mysql

Scripts de inicialização:	init.sql
---------------------------	----------

Container: Frontend (se separado)

Imagem base:	(Não utilizado)
Variáveis de ambiente:	Não tem
Volumes / arquivos:	(Não utilizado)
Observações:	(---)

4. FLUXO DE COMUNICAÇÃO ENTRE SERVIÇOS

#	Origem	Destino	Protocolo / Porta	Descrição
1	Navegador	Nginx	HTTP :80	Requisição do usuário
2	Nginx	API Backend	HTTP interno	Encaminha as requisições para a API
3	API Backend	MySQL	TCP :3306	Cadastro dos dados
4	MySQL	API Backend	TCP :3306	Retorno dos dados
5	API Backend	Nginx	HTTP interno	JSON de resposta
6	Nginx	Navegador	HTTP :80	Resposta final ao usuário

Observações / Fluxo adicional:

5. CONFIGURAÇÃO DO NGINX

Conteúdo do nginx.conf:

```
server {
    listen 80;
    server_name localhost;

    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass http://api:5000/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Explicação da configuração:

Server_name: Nome do servidor que vai enviar os usuários até o front-end da aplicação

Listen: Indica qual porta será escutada pelo nginx

Proxy_pass: Indica qual a porta que a aplicação se situa para que o proxy reverso estabeleça a comunicação com o site.

6. ANÁLISE DE TRÁFEGO – WIRESHARK

#	Protocolo	IP Origem	IP Destino	Porta	Descrição / Observação
1	HTTP	192.168.1.74	95.216.195.133	80	GET /nm-check.txt HTTP/1.1
2	HTTP/JSON	127.0.0.1	127.0.0.1	80	POST /api/users HTTP/1.1, JSON (application/json)
3	HTTP	127.0.0.1	127.0.0.1	80	GET /api/users HTTP/1.1

The image displays a Wireshark packet capture analysis. The top section shows a list of captured packets with columns for No., Time, Delta, Source, Destination, Protocol, Length, and Info. The selected packet (No. 2219) is a GET request for /nm-check.txt from 192.168.1.74 to 95.216.195.133. The bottom section shows the packet details for the selected packet, including the Ethernet II header, Internet Protocol Version 4 header, and Hypertext Transfer Protocol header. The packet is 156 bytes long and is a GET request for /nm-check.txt.

Frame 2219: Packet, 156 bytes on wire (1248 bits), 156 bytes captured (1248 bits) on
Linux cooked capture v1
Internet Protocol Version 4, Src: 192.168.1.74, Dst: 95.216.195.133
Transmission Control Protocol, Src Port: 57282, Dst Port: 80, Seq: 1, Ack: 8
Hypertext Transfer Protocol

Frame (frame), 156 bytes
Pacotes: 8806 - Exibidos: 16 (0.2%)
Perfil: Default

No. Time Delta Source Destination Protocol Length Info

2876 388.413579315 0.001417453 192.168.1.74 199.232.13.91 HTTP 378 GET /success.txt?ipv4 HTTP/1.1

6752 447.356011305 58.942431990 2804:d59:aa36... 2a04:4e42:40::... HTTP 381 GET /canonical.html HTTP/1.1

6764 447.412291415 0.056280110 2804:d59:aa36... 2a04:4e42:40::... HTTP 398 GET /success.txt?ipv6 HTTP/1.1

6767 447.416785167 0.004493752 192.168.1.74 199.232.13.91 HTTP 378 GET /success.txt?ipv4 HTTP/1.1

7174 531.161806940 83.745021773 192.168.1.74 95.216.195.133 HTTP 156 GET /nm-check.txt HTTP/1.1

7664 816.243605485 285.081878545 192.168.1.74 95.216.195.133 HTTP 402 GET / HTTP/1.1

7670 816.671819475 0.428133990 192.168.1.74 95.216.195.133 HTTP 419 GET /favicon.ico HTTP/1.1

7716 831.137680115 14.465860640 192.168.1.74 95.216.195.133 HTTP 156 GET /nm-check.txt HTTP/1.1

8884 1117.3510960... 286.213407975 127.0.0.1 127.0.0.1 HTTP 445 GET /api/users HTTP/1.1

8932 1131.1692864... 13.818190346 192.168.1.74 95.216.195.133 HTTP 156 GET /nm-check.txt HTTP/1.1

9056 1217.5172468... 86.347960369 127.0.0.1 127.0.0.1 HTTP/JSON 568 POST /api/users HTTP/1.1 , JSON (application/json)

9060 1217.5704681... 0.053221326 127.0.0.1 127.0.0.1 HTTP 445 GET /api/users HTTP/1.1

Frame 9056: Packet, 568 bytes on wire (4544 bits), 568 bytes captured (4544 bits)
Linux cooked capture v1
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
0100 = Version: 4
.... 0101 = Header Length: 20 bytes (5)
Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 552
Identification: 0x3eae (16046)
010 = Flags: 0x2, Don't fragment
...0 0000 0000 0000 = Fragment Offset: 0

Bytes 84-91: Request Version (http.request.version)
Pacotes: 9273 - Exibidos: 20 (0.2%)
Perfil: Default

Interpretação geral do tráfego capturado:

As capturas demonstram o funcionamento da API de usuários. Por exemplo, quando o usuário aperta no botão “atualizar”, da lista de usuários, a API faz uma requisição GET para trazer a lista de usuários registrada no banco de dados da aplicação. Além disso, foi observado que a requisição POST, usa o protocolo HTTP/JSON para que a API consiga interagir com o backend da aplicação, que realizará o cadastro no banco de dados.

7. LINK DO REPOSITÓRIO GIT

URL do Repositório Git:	APIProjeto
Plataforma (GitHub / GitLab / etc.):	GitHub
Branch principal:	main
README.md incluído?	Sim

8. LINK DO VÍDEO DEMONSTRATIVO

URL do Vídeo:	https://youtu.be/4SHrHbbHyGY
Plataforma (YouTube / Drive / etc.):	Youtube
Duração do Vídeo:	1 min 40 seg